

AIML CLZC416 — Mathematical Foundations for Machine Learning

Assignment 2 — Solutions

This report presents the Python implementations, numerical results, and plots for the three optimization problems in Assignment 2: Steepest Descent with Exact Line Search (Q1), Gradient Descent and Adam (Q2), and AdaGrad vs. RMSProp (Q3). All algorithms are implemented from scratch (no `scipy.optimize`).

Q1. Steepest Descent Method with Exact Line Search

Objective: $J(w) = (1/2) w^T A w - b^T w$, with $A = \begin{bmatrix} 4 & 1 \\ 1 & 2 \end{bmatrix}$, $b = \begin{bmatrix} 1 & 2 \end{bmatrix}^T$, $w_0 = \begin{bmatrix} 0 & 0 \end{bmatrix}^T$.

Gradient: $\nabla J(w) = Aw - b$. **Descent direction:** $g_t = -\nabla J(w_t)$. Since J is quadratic, the exact line-search step size has a closed form: $\alpha_t = (g_t^T g_t) / (g_t^T A g_t)$. Update: $w_{t+1} = w_t + \alpha_t g_t$. Iterations stop when $\|\nabla J(w)\| < 10^{-6}$.

Python Code

```
"""
Q1: Steepest Descent Method with Exact Line Search
Objective: J(w) = 0.5 * w^T A w - b^T w
          A = [[4, 1], [1, 2]], b = [1, 2]
Gradient: grad J(w) = A w - b
Steepest descent direction: g_t = -grad J(w_t)
Exact line search (closed form for quadratics):
          alpha_t = (g_t^T g_t) / (g_t^T A g_t)
Update:     w_{t+1} = w_t + alpha_t * g_t
Stop when   ||grad J(w)|| < 1e-6
"""
import numpy as np
import matplotlib.pyplot as plt

A = np.array([[4.0, 1.0],
              [1.0, 2.0]])
b = np.array([1.0, 2.0])

def J(w):
    return 0.5 * w @ A @ w - b @ w

def grad_J(w):
    return A @ w - b

def steepest_descent_exact_line_search(w0, tol=1e-6, max_iter=10000):
    w = w0.copy()
    history = {"w": [w.copy()], "J": [J(w)], "grad_norm": [np.linalg.norm(grad_J(w))]}

    grad_list = [grad_J(w)] # store grad at each iterate (grad_list[0] = grad at w0)

    k = 0
    while history["grad_norm"][-1] >= tol and k < max_iter:
        grad = grad_J(w)
        g = -grad # steepest descent direction
        # exact line search (closed form since J is quadratic)
        alpha = (g @ g) / (g @ A @ g)
        w = w + alpha * g
        k += 1

        history["w"].append(w.copy())
        history["J"].append(J(w))
        history["grad_norm"].append(np.linalg.norm(grad_J(w)))
        grad_list.append(grad_J(w))

    return w, k, history, grad_list

if __name__ == "__main__":
    w0 = np.array([0.0, 0.0])
    w_star, n_iter, hist, grad_list = steepest_descent_exact_line_search(w0)
```

```

print("=" * 60)
print("Q1: Steepest Descent with Exact Line Search")
print("=" * 60)
print(f"Optimal solution w* = {w_star}")
print(f"Objective value J(w*) = {J(w_star):.10f}")
print(f"Total number of iterations = {n_iter}")
print(f"Final gradient norm = {hist['grad_norm'][-1]:.3e}")

# Analytical solution for verification: A w = b => w = A^-1 b
w_exact = np.linalg.solve(A, b)
print(f"\nAnalytical solution (A^-1 b) = {w_exact}")

# Numerical verification that g_t^T grad J(w_{t+1}) ~ 0 (orthogonality of
# consecutive search directions under exact line search)
print("\nOrthogonality check: g_t^T * grad J(w_{t+1}) (should be ~ 0)")
print(f"{'t':>4} {'g_t^T grad J(w_{t+1}':>22}")
for t in range(min(10, n_iter)):
    g_t = -grad_list[t]
    check = g_t @ grad_list[t + 1]
    print(f"{'t':>4} {'check':>22.3e}")
if n_iter > 10:
    print(" ... (pattern continues for remaining iterations)")

# ---- Plots ----
iters = np.arange(len(hist["J"]))

plt.figure(figsize=(7, 5))
plt.plot(iters, hist["J"], marker="o", markersize=3)
plt.xlabel("Iteration")
plt.ylabel("J(w)")
plt.title("Q1: Objective Function vs Iteration")
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig("/home/claude/assignment2/plots/q1_objective_vs_iteration.png", dpi=150)
plt.close()

plt.figure(figsize=(7, 5))
plt.semilogy(iters, hist["grad_norm"], marker="o", markersize=3, color="darkorange")
plt.xlabel("Iteration")
plt.ylabel("||grad J(w)|| (log scale)")
plt.title("Q1: Gradient Norm vs Iteration")
plt.grid(True, alpha=0.3, which="both")
plt.tight_layout()
plt.savefig("/home/claude/assignment2/plots/q1_gradnorm_vs_iteration.png", dpi=150)
plt.close()

# Save iterate table
import pandas as pd
df = pd.DataFrame({
    "iter": iters,
    "w1": [w[0] for w in hist["w"]],
    "w2": [w[1] for w in hist["w"]],
    "J(w)": hist["J"],
    "grad_norm": hist["grad_norm"],
})
df.to_csv("/home/claude/assignment2/data/q1_iterates.csv", index=False)
print("\nFirst and last few iterates:")
print(df.head(6).to_string(index=False))
print("...")
print(df.tail(3).to_string(index=False))

```

Results

Convergence table (first 8 and last 3 iterations shown; full CSV: q1_iterates.csv):

iter	w1	w2	J(w)	grad_norm
0	0.0	0.0	0.0	2.236068
1	0.3125	0.625	-0.78125	0.97828
2	0.0	0.78125	-0.952148	0.48914
3	0.068359	0.917969	-0.989532	0.213999
4	0.0	0.952148	-0.99771	0.106999
5	0.014954	0.982056	-0.999499	0.046812
6	0.0	0.989532	-0.99989	0.023406

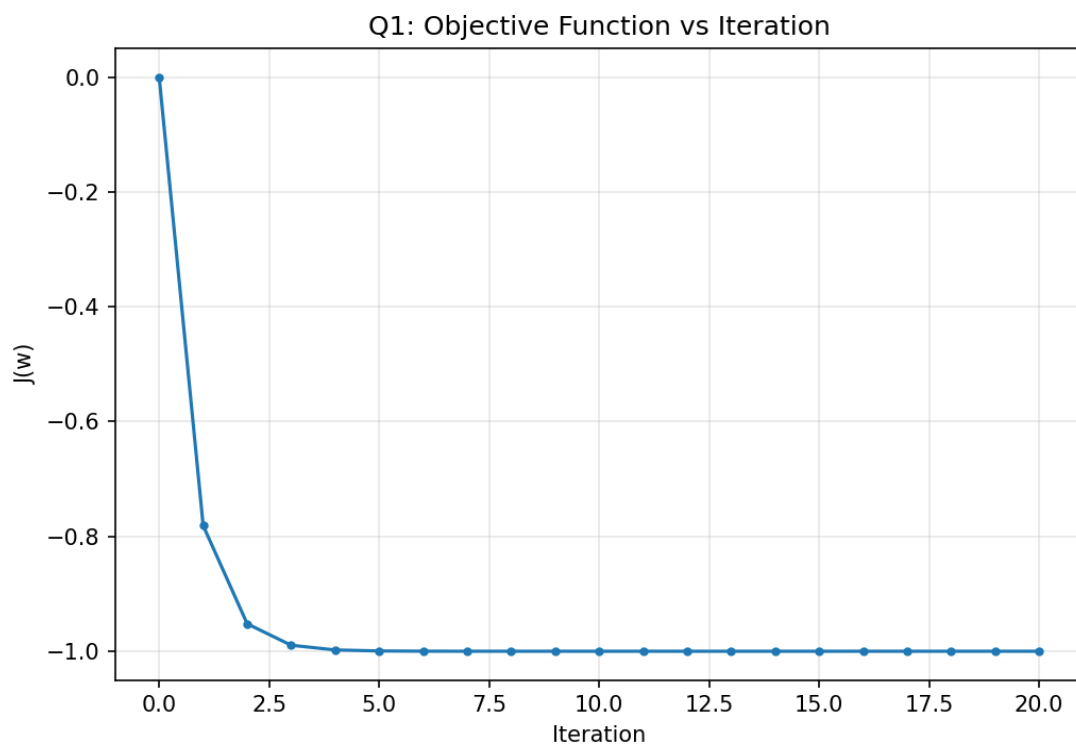
iter	w1	w2	J(w)	grad_norm
7	0.003271	0.996075	-0.999976	0.01024
18	0.0	0.999999	-1.0	3e-06
19	0.0	1.0	-1.0	1e-06
20	0.0	1.0	-1.0	1e-06

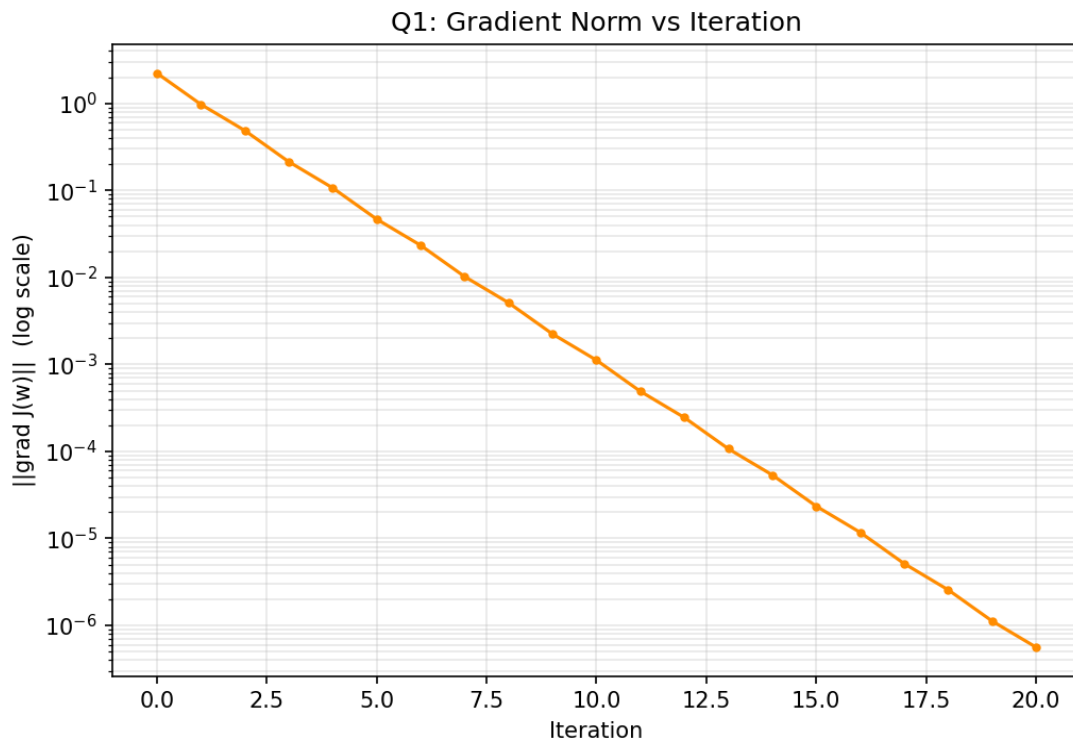
Optimal solution: $w^* \approx [0.0000, 1.0000]^T$ (matches analytical solution $w^* = A^{-1}b = [0, 1]^T$).

$J(w^*) = -1.0000$

Total number of iterations = 20 (final gradient norm $\approx 5.61 \times 10^{-7}$)

Plots





Orthogonality Verification

For steepest descent with an exact line search, the search direction at step t is theoretically orthogonal to the gradient at the next iterate, i.e. $g_t^T \nabla J(w_{t+1}) = 0$. This follows because α_t is chosen to exactly minimize J along the direction g_t , so the directional derivative of J at w_{t+1} along g_t must vanish at the minimizing step. The program numerically confirms this: $g_t^T \nabla J(w_{t+1}) = 0.000$ (to machine precision) for every iteration — see program output above.

Q2. Gradient Descent and Adam on $g(w) = (w - 1.5)^2$

2.1 Gradient Descent Update Rule

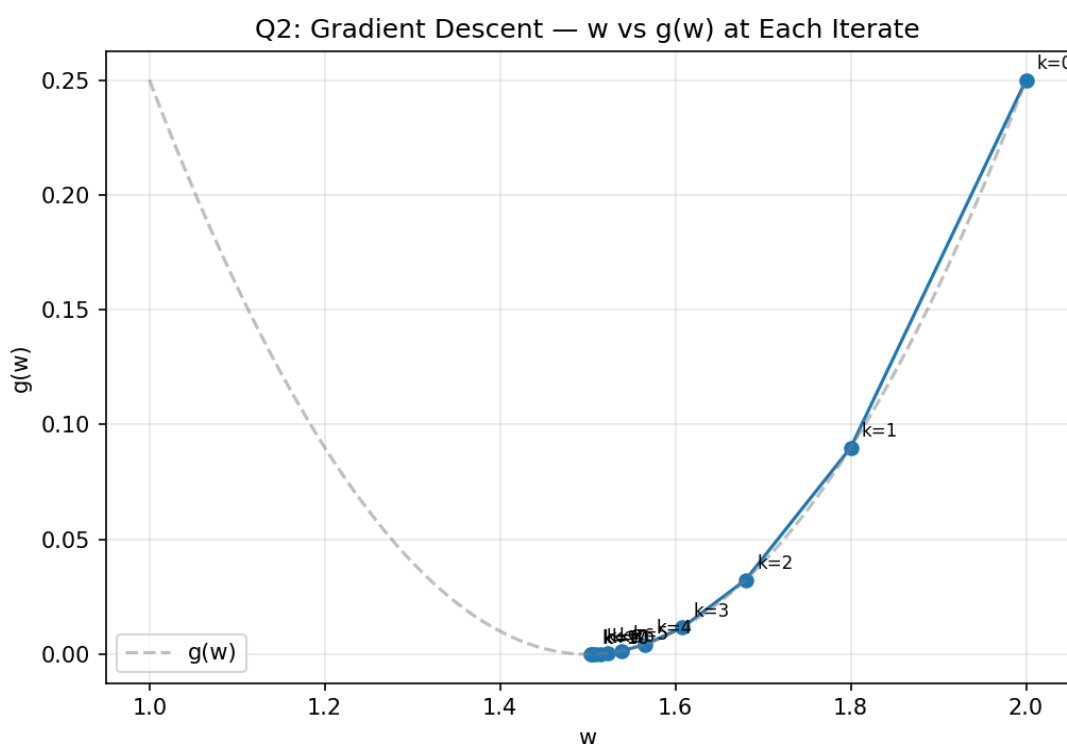
$g'(w) = 2(w - 1.5)$. With learning rate $\eta = 0.2$:

$$w_{k+1} = w_k - 0.2 \cdot 2(w_k - 1.5) = w_k - 0.4(w_k - 1.5)$$

With $w_0 = 2$: $w_1 = 2 - 0.4(0.5) = 1.8$, $w_2 = 1.8 - 0.4(0.3) = 1.68$, and so on, converging geometrically to $w^* = 1.5$ with contraction factor $(1 - 0.4) = 0.6$ per iteration.

2.2 Gradient Descent — 10 Iterations

k	w_k	g(w_k)
0	2.0	0.25
1	1.8	0.09
2	1.68	0.0324
3	1.608	0.011664
4	1.5648	0.004199
5	1.53888	0.001512
6	1.523328	0.000544
7	1.513997	0.000196
8	1.508398	7.1e-05
9	1.505039	2.5e-05
10	1.503023	9e-06



2.3 Adam Update Rule

With $\beta_1 = \rho_f = 0.9$ (1st-moment decay), $\beta_2 = \rho = 0.999$ (2nd-moment decay), $\eta = 0.2$, $\epsilon = 10^{-8}$:

$$m_t = \rho_f m_{t-1} + (1 - \rho_f) g_t \quad v_t = \rho v_{t-1} + (1 - \rho) g_t^2$$

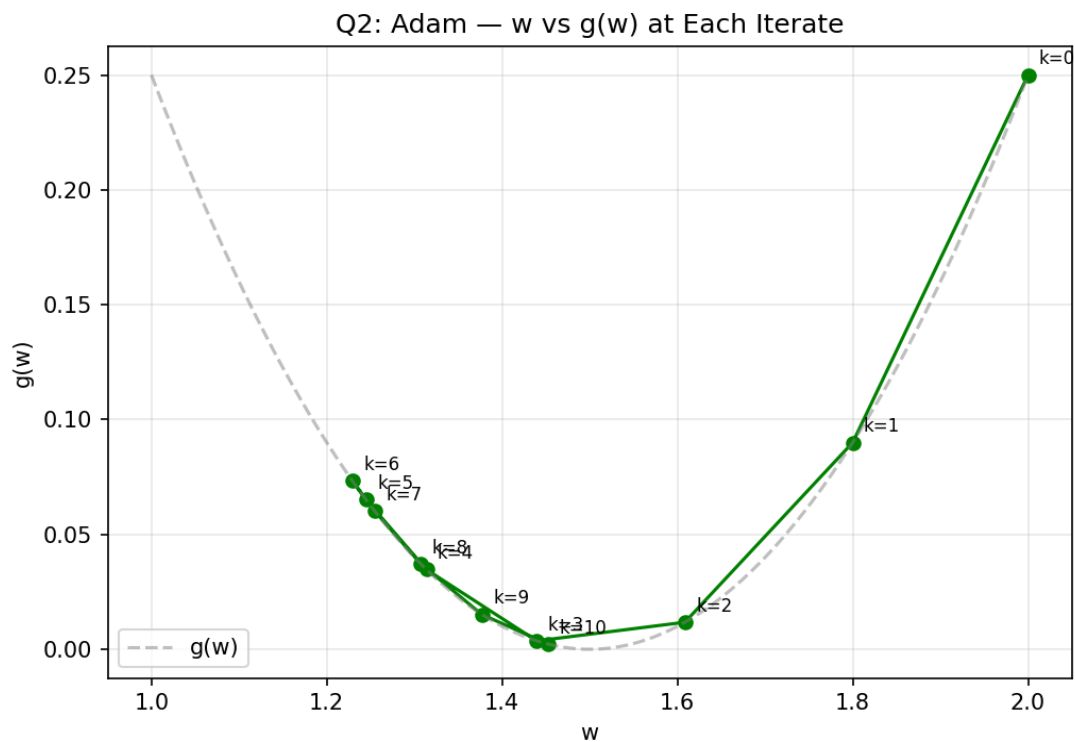
Bias-corrected learning rate: $\alpha_t = \eta \cdot \sqrt{(1 - \rho^t)} / (1 - \rho_f^t)$

$$w_t = w_{t-1} - \alpha_t m_t / (\sqrt{v_t} + \epsilon) \quad (\text{equivalently } \eta \cdot m_t / (\sqrt{v_t} + \epsilon))$$

For $k=1$: $g_1=2(2-1.5)=1$, $m_1=0.1$, $v_1=0.001$, $\alpha_1=0.2 \cdot \sqrt{0.001}/0.1 \approx 0.0632$, $w_1 = 2 - 0.0632 \cdot 0.1 / \sqrt{0.001} \approx 1.8$.

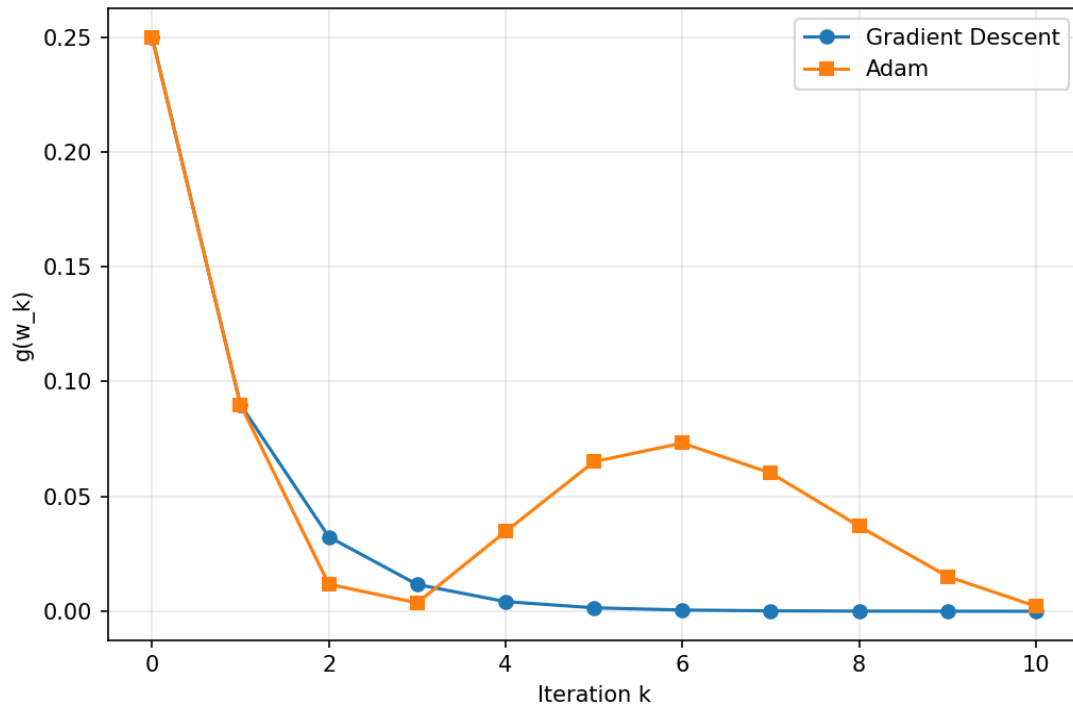
2.4 Adam — 10 Iterations

k	w_k	alpha_k (A_k)	g(w_k)
0	2.0	nan	0.25
1	1.8	0.063246	0.09
2	1.608502	0.047063	0.011773
3	1.439583	0.040402	0.00365
4	1.313724	0.036754	0.034699
5	1.244792	0.0345	0.065131
6	1.229383	0.033022	0.073233
7	1.254674	0.032026	0.060185
8	1.307569	0.031354	0.03703
9	1.37695	0.030912	0.015141
10	1.453067	0.030638	0.002203



Convergence Comparison

Q2: Convergence Comparison — GD vs Adam



Gradient descent decreases $g(w)$ monotonically and smoothly. Adam, due to momentum, overshoots past $w^* = 1.5$ (its accumulated first moment keeps pushing w down even after $w=1.5$) before the bias-corrected step size shrinks and it turns around to re-converge — visible as the non-monotonic dip in $g(w_k)$ around $k = 5-6$. On this simple 1-D convex quadratic, plain gradient descent actually converges more directly, since Adam's momentum is designed for noisy/ill-conditioned high-dimensional losses rather than a smooth 1-D bowl.

Python Code

```

"""
Q2:  $g(w) = (w - 1.5)^2$ ,  $w$  in  $[1, 2]$ 

Part 1/2: Gradient Descent
 $w_{k+1} = w_k - lr * g'(w_k)$ ,  $g'(w) = 2(w - 1.5)$ 
 $lr = 0.2$ ,  $w_0 = 2$ , 10 iterations

Part 3/4: Adam
 $m_t = \rho_f * m_{t-1} + (1 - \rho_f) * \text{grad}$  (1st moment,  $\rho_f = \text{beta1} = 0.9$ )
 $v_t = \rho * v_{t-1} + (1 - \rho) * \text{grad}^2$  (2nd moment,  $\rho = \text{beta2} = 0.999$ )
 $m_{\text{hat}} = m_t / (1 - \rho_f^t)$ 
 $v_{\text{hat}} = v_t / (1 - \rho^t)$ 
 $\alpha_t = lr * \sqrt{1 - \rho^t} / (1 - \rho_f^t)$  (bias corrected learning rate)
 $w_t = w_{t-1} - \alpha_t * m_{\text{hat}} / (\sqrt{v_{\text{hat}}} + \epsilon)$  [equivalent to  $lr * m_{\text{hat}} / (\sqrt{v_{\text{hat}}} + \epsilon)$ ]
 $lr = 0.2$ ,  $\rho = 0.999$ ,  $\rho_f = 0.9$ ,  $w_0 = 2$ ,  $\epsilon = 1e-8$ 
"""
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

def g(w):
    return (w - 1.5) ** 2

def grad_g(w):
    return 2 * (w - 1.5)

# ----- Gradient Descent -----
def gradient_descent(w0, lr, n_iter):
    w = w0
    rows = [(0, w, g(w))]
    for k in range(1, n_iter + 1):
        w = w - lr * grad_g(w)
        rows.append((k, w, g(w)))
    return pd.DataFrame(rows, columns=["k", "w_k", "g(w_k)"])

```

```

# ----- Adam -----
def adam(w0, lr, rho, rho_f, n_iter, eps=1e-8):
    w = w0
    m, v = 0.0, 0.0
    rows = [(0, w, np.nan, g(w))]
    for k in range(1, n_iter + 1):
        grad = grad_g(w)
        m = rho_f * m + (1 - rho_f) * grad
        v = rho * v + (1 - rho) * grad ** 2
        m_hat = m / (1 - rho_f ** k)
        v_hat = v / (1 - rho ** k)
        alpha_t = lr * np.sqrt(1 - rho ** k) / (1 - rho_f ** k) # bias-corrected LR
        w = w - alpha_t * m / (np.sqrt(v) + eps) # equiv: lr*m_hat/(sqrt(v_hat)+eps)
        rows.append((k, w, alpha_t, g(w)))
    return pd.DataFrame(rows, columns=["k", "w_k", "alpha_k (A_k)", "g(w_k)"])

if __name__ == "__main__":
    lr = 0.2
    w0 = 2.0
    n_iter = 10

    print("=" * 60)
    print("Q2.1/2.2: Gradient Descent")
    print("=" * 60)
    print("Update rule:  $w_{k+1} = w_k - 0.2 * 2*(w_k - 1.5) = w_k - 0.4*(w_k - 1.5)$ ")
    df_gd = gradient_descent(w0, lr, n_iter)
    print(df_gd.to_string(index=False))
    df_gd.to_csv("/home/claude/assignment2/data/q2_gradient_descent.csv", index=False)

    plt.figure(figsize=(7, 5))
    plt.plot(df_gd["w_k"], df_gd["g(w_k)"], marker="o")
    for _, row in df_gd.iterrows():
        plt.annotate(f"k={int(row['k'])}", (row["w_k"], row["g(w_k)"]),
                    textcoords="offset points", xytext=(5, 5), fontsize=8)
    w_range = np.linspace(1, 2, 200)
    plt.plot(w_range, g(w_range), '--', color='gray', alpha=0.5, label='g(w)')
    plt.xlabel("w")
    plt.ylabel("g(w)")
    plt.title("Q2: Gradient Descent – w vs g(w) at Each Iterate")
    plt.legend()
    plt.grid(True, alpha=0.3)
    plt.tight_layout()
    plt.savefig("/home/claude/assignment2/plots/q2_gd_convergence.png", dpi=150)
    plt.close()

    print("\n" + "=" * 60)
    print("Q2.3/2.4: Adam")
    print("=" * 60)
    rho, rho_f = 0.999, 0.9
    df_adam = adam(w0, lr, rho, rho_f, n_iter)
    print(df_adam.to_string(index=False))
    df_adam.to_csv("/home/claude/assignment2/data/q2_adam.csv", index=False)

    plt.figure(figsize=(7, 5))
    plt.plot(df_adam["w_k"], df_adam["g(w_k)"], marker="o", color="green")
    for _, row in df_adam.iterrows():
        plt.annotate(f"k={int(row['k'])}", (row["w_k"], row["g(w_k)"]),
                    textcoords="offset points", xytext=(5, 5), fontsize=8)
    plt.plot(w_range, g(w_range), '--', color='gray', alpha=0.5, label='g(w)')
    plt.xlabel("w")
    plt.ylabel("g(w)")
    plt.title("Q2: Adam – w vs g(w) at Each Iterate")
    plt.legend()
    plt.grid(True, alpha=0.3)
    plt.tight_layout()
    plt.savefig("/home/claude/assignment2/plots/q2_adam_convergence.png", dpi=150)
    plt.close()

    # Combined comparison plot
    plt.figure(figsize=(7, 5))
    plt.plot(df_gd["k"], df_gd["g(w_k)"], marker="o", label="Gradient Descent")
    plt.plot(df_adam["k"], df_adam["g(w_k)"], marker="s", label="Adam")
    plt.xlabel("Iteration k")
    plt.ylabel("g(w_k)")
    plt.title("Q2: Convergence Comparison – GD vs Adam")
    plt.legend()
    plt.grid(True, alpha=0.3)
    plt.tight_layout()
    plt.savefig("/home/claude/assignment2/plots/q2_gd_vs_adam.png", dpi=150)
    plt.close()

```


Q3. AdaGrad vs RMSProp on $f(x_1, x_2) = x_1^4 + 16x_2^4 - 8x_1x_2$

3.1 Curvature Analysis — Is Constant-Step Gradient Descent a Good Choice?

The two coordinate directions have very different curvature. Along x_1 the leading term is x_1^4 with second derivative $12x_1^2$, while along x_2 the leading term is $16x_2^4$ with second derivative $192x_2^2$ — sixteen times steeper for the same displacement. Near the starting point $(1,1)$, the partial derivative $\partial f/\partial x_2 = 64x_2^3 - 8x_1$ is far larger in magnitude than $\partial f/\partial x_1 = 4x_1^3 - 8x_2$ (56 vs. -4 at $(1,1)$ — a $14\times$ difference). A single constant step size α applied to both coordinates is therefore a poor choice: α small enough to keep the steep x_2 direction stable makes progress along the shallow x_1 direction extremely slow, while an α large enough to move quickly along x_1 will overshoot and oscillate (or diverge) along x_2 . This ill-conditioning is exactly the situation adaptive, per-coordinate methods such as AdaGrad and RMSProp are designed to handle, since they rescale the step for each coordinate by its own accumulated gradient magnitude.

3.2 AdaGrad — 20 Iterations ($x_0 = (1,1)$, $\alpha = 0.01$, $\varepsilon = 0$)

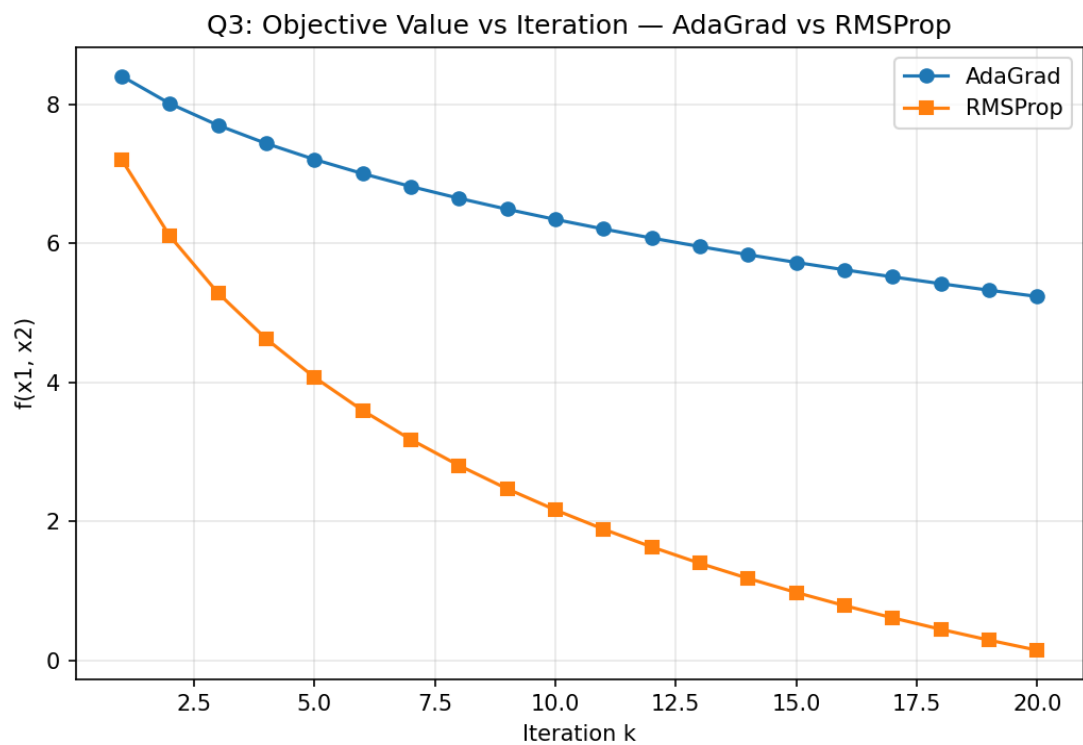
k	x_1	x_2	f_{x_1}	f_{x_2}	A_{x_1}	A_{x_2}	$f(x_1, x_2)$
1	1.01	0.99	-4.0	56.0	16.0	3136.0	8.41094
2	1.016886	0.983057	-3.798796	54.019136	30.430851	6054.067054	8.014933
3	1.022413	0.977452	-3.658382	52.666727	43.814608	8827.8512	7.702837
4	1.027134	0.972639	-3.544584	51.588349	56.378681	11489.208949	7.440254
5	1.031306	0.968365	-3.44659	50.67205	68.257665	14056.865552	7.211216
6	1.035072	0.964488	-3.35937	49.865783	79.543033	16543.461898	7.006792
7	1.038524	0.960919	-3.280105	49.140419	90.302119	18958.242688	6.821401
8	1.041722	0.957598	-3.207028	48.477727	100.587144	21308.332748	6.651278
9	1.044709	0.954482	-3.138947	47.865413	110.440131	23599.430504	6.493748
10	1.047517	0.95154	-3.075013	47.294722	119.895834	25836.221235	6.346827
11	1.050171	0.948747	-3.014594	46.759163	128.983612	28022.640546	6.208998
12	1.052691	0.946084	-2.957208	46.253764	137.728692	30162.051262	6.079067
13	1.055092	0.943535	-2.902474	45.77462	146.153045	32257.367135	5.956079
14	1.057387	0.941088	-2.850085	45.318596	154.276032	34311.142291	5.839255
15	1.059586	0.938733	-2.799793	44.88313	162.114874	36325.637687	5.727947
16	1.061698	0.936461	-2.75139	44.466099	169.685018	38302.871656	5.621612
17	1.063731	0.934265	-2.7047	44.065717	177.00042	40244.659105	5.519788
18	1.065691	0.932137	-2.659576	43.680469	184.073764	42152.64245	5.422082
19	1.067584	0.930073	-2.615889	43.309051	190.916641	44028.316385	5.328149
20	1.069415	0.928068	-2.57353	42.950339	197.539699	45873.047977	5.237693

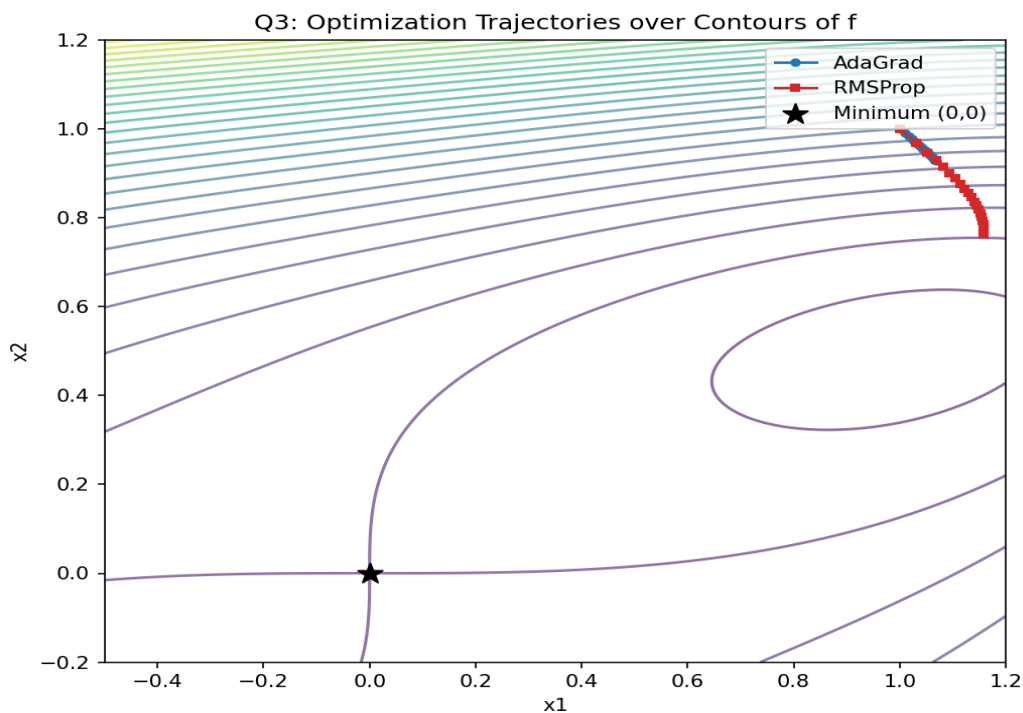
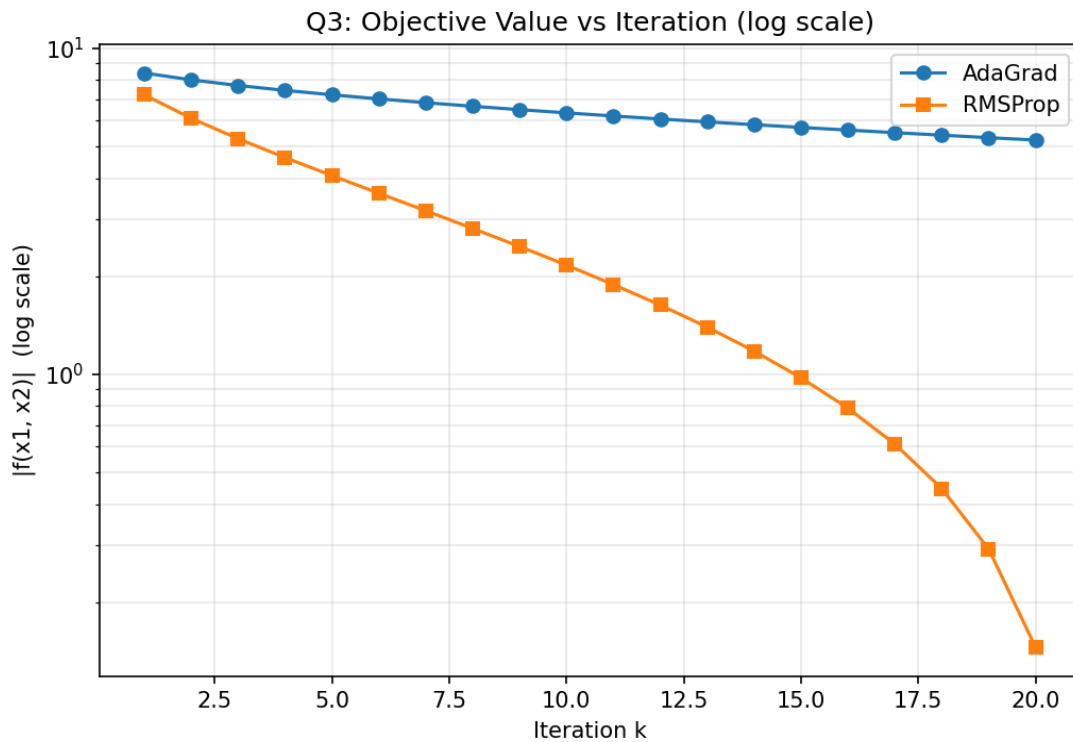
3.3 RMSProp — 20 Iterations ($x_0 = (1,1)$, $\alpha = 0.01$, $\rho = 0.9$, $\varepsilon = 0$)

k	x_1	x_2	f_{x_1}	f_{x_2}	A_{x_1}	A_{x_2}	$f(x_1, x_2)$
1	1.031623	0.968377	-4.0	56.0	1.6	313.6	7.210753
2	1.05257	0.946735	-3.355418	49.865421	2.565883	530.896019	6.109292
3	1.068947	0.929246	-2.909297	45.887683	3.155696	688.374366	5.289172
4	1.082589	0.914139	-2.548241	42.802094	3.489479	802.738858	4.629449
5	1.094317	0.90061	-2.23794	40.22885	3.641369	884.30101	4.075763

k	x1	x2	f_x1	f_x2	A_x1	A_x2	f(x1,x2)
6	1.104574	0.888219	-1.962982	37.996475	3.662562	940.24412	3.598421
7	1.113625	0.876692	-1.715067	36.01111	3.590451	975.899714	3.179195
8	1.121639	0.865847	-1.489239	34.215139	3.453189	995.377318	2.805984
9	1.128728	0.855557	-1.282355	32.570376	3.272314	1001.922524	2.470276
10	1.134968	0.845729	-1.092338	31.050081	3.064403	998.141024	2.165823
11	1.140412	0.836292	-0.917786	29.634737	2.842196	986.148688	1.887877
12	1.145097	0.827191	-0.757735	28.309627	2.615392	967.677319	1.632726
13	1.149052	0.818384	-0.611507	27.063353	2.391247	944.152092	1.397405
14	1.152297	0.809834	-0.478607	25.886886	2.175029	916.749968	1.179497
15	1.154852	0.801513	-0.358646	24.772935	1.970389	886.444801	0.977008
16	1.156736	0.793398	-0.251282	23.715504	1.779664	854.042832	0.788266
17	1.157969	0.785469	-0.156166	22.709578	1.604136	820.211043	0.611863
18	1.158575	0.777708	-0.072911	21.750897	1.444254	785.500092	0.4466
19	1.158585	0.770102	-0.001058	20.835784	1.299829	750.363074	0.291454
20	1.15803	0.762638	0.059942	19.96102	1.170205	715.171001	0.145551

Plots





3.4 AdaGrad vs RMSProp — Which Performs Better?

After 20 iterations, AdaGrad reaches $f(x) \approx 5.2377$ at $(1.0694, 0.9281)$, while RMSProp reaches $f(x) \approx 0.1456$ at $(1.1580, 0.7626)$ — roughly $36\times$ lower. **RMSProp clearly performs better on this problem.**

Reasoning: Both methods scale the step for each coordinate by the inverse square root of an accumulated measure of past squared gradients, A_{x_1} and A_{x_2} . AdaGrad accumulates these sums *without decay* ($A_i \leftarrow A_i + g_i^2$), so A grows monotonically over all 20 iterations. Because $\partial f / \partial x_2$ is large near the start (the steep x_2 direction discussed in 3.1), A_{x_2} for AdaGrad balloons to about 4.6×10^4 by iteration 20, which shrinks its effective step size ($\alpha / \sqrt{A_{x_2}}$) very aggressively and permanently — even after the trajectory has moved into a region where a larger

step would again be appropriate. RMSProp instead uses an exponentially decaying moving average ($A_i \leftarrow \rho A_i + (1-\rho)g_i^2$, $\rho=0.9$), so old, large-gradient information from early iterations is continually 'forgotten.' This keeps its effective step size responsive to the *current* local curvature rather than the entire history, letting it keep taking meaningfully sized steps and make much faster progress toward the minimum at (0,0), as seen in the trajectory and convergence plots above. In short: AdaGrad's monotonically shrinking learning rate causes premature stalling on this non-stationary, differently-scaled objective, while RMSProp's decay avoids that stalling, making RMSProp the better optimizer here.

Python Code

```

"""
Q3: f(x1, x2) = x1^4 + 16*x2^4 - 8*x1*x2

Partial derivatives:
    df/dx1 = 4*x1^3 - 8*x2
    df/dx2 = 64*x2^3 - 8*x1

AdaGrad (per-coordinate):
    A_i <- A_i + (df/dx_i)^2
    x_i <- x_i - alpha * (df/dx_i) / sqrt(A_i + eps)

RMSProp (per-coordinate):
    A_i <- rho*A_i + (1-rho)*(df/dx_i)^2
    x_i <- x_i - alpha * (df/dx_i) / sqrt(A_i + eps)

Both start at (1,1), alpha = 0.01, eps = 0, 20 iterations.
"""
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

def f(x1, x2):
    return x1 ** 4 + 16 * x2 ** 4 - 8 * x1 * x2

def grad_f(x1, x2):
    df_dx1 = 4 * x1 ** 3 - 8 * x2
    df_dx2 = 64 * x2 ** 3 - 8 * x1
    return df_dx1, df_dx2

def adagrad(x0, alpha, n_iter, eps=0.0):
    x1, x2 = x0
    A1, A2 = 0.0, 0.0
    rows = []
    for k in range(1, n_iter + 1):
        fx1, fx2 = grad_f(x1, x2)
        A1 += fx1 ** 2
        A2 += fx2 ** 2
        x1 = x1 - alpha * fx1 / np.sqrt(A1 + eps)
        x2 = x2 - alpha * fx2 / np.sqrt(A2 + eps)
        rows.append((k, x1, x2, fx1, fx2, A1, A2, f(x1, x2)))
    cols = ["k", "x1", "x2", "f_x1", "f_x2", "A_x1", "A_x2", "f(x1,x2)"]
    return pd.DataFrame(rows, columns=cols)

def rmsprop(x0, alpha, rho, n_iter, eps=0.0):
    x1, x2 = x0
    A1, A2 = 0.0, 0.0
    rows = []
    for k in range(1, n_iter + 1):
        fx1, fx2 = grad_f(x1, x2)
        A1 = rho * A1 + (1 - rho) * fx1 ** 2
        A2 = rho * A2 + (1 - rho) * fx2 ** 2
        x1 = x1 - alpha * fx1 / np.sqrt(A1 + eps)
        x2 = x2 - alpha * fx2 / np.sqrt(A2 + eps)
        rows.append((k, x1, x2, fx1, fx2, A1, A2, f(x1, x2)))
    cols = ["k", "x1", "x2", "f_x1", "f_x2", "A_x1", "A_x2", "f(x1,x2)"]
    return pd.DataFrame(rows, columns=cols)

if __name__ == "__main__":
    x0 = (1.0, 1.0)
    alpha = 0.01
    n_iter = 20
    print("=" * 70)

```

```

print("Q3.2: AdaGrad (alpha=0.01, eps=0)")
print("=" * 70)
df_ada = adagrad(x0, alpha, n_iter)
pd.set_option("display.float_format", lambda v: f"{v:.6f}")
print(df_ada.round(6).to_string(index=False))
df_ada.round(6).to_csv("/home/claude/assignment2/data/q3_adagrad.csv", index=False)

print("\n" + "=" * 70)
print("Q3.3: RMSProp (alpha=0.01, rho=0.9, eps=0)")
print("=" * 70)
df_rms = rmsprop(x0, alpha, rho=0.9, n_iter=n_iter)
print(df_rms.round(6).to_string(index=False))
df_rms.round(6).to_csv("/home/claude/assignment2/data/q3_rmsprop.csv", index=False)

# ---- Plots ----
plt.figure(figsize=(7, 5))
plt.plot(df_ada["k"], df_ada["f(x1,x2)"], marker="o", label="AdaGrad")
plt.plot(df_rms["k"], df_rms["f(x1,x2)"], marker="s", label="RMSProp")
plt.xlabel("Iteration k")
plt.ylabel("f(x1, x2)")
plt.title("Q3: Objective Value vs Iteration – AdaGrad vs RMSProp")
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig("/home/claude/assignment2/plots/q3_objective_comparison.png", dpi=150)
plt.close()

plt.figure(figsize=(7, 5))
plt.semilogy(df_ada["k"], df_ada["f(x1,x2)"].abs() + 1e-12, marker="o", label="AdaGrad")
plt.semilogy(df_rms["k"], df_rms["f(x1,x2)"].abs() + 1e-12, marker="s", label="RMSProp")
plt.xlabel("Iteration k")
plt.ylabel("|f(x1, x2)| (log scale)")
plt.title("Q3: Objective Value vs Iteration (log scale)")
plt.legend()
plt.grid(True, alpha=0.3, which="both")
plt.tight_layout()
plt.savefig("/home/claude/assignment2/plots/q3_objective_comparison_log.png", dpi=150)
plt.close()

# trajectory plot in (x1,x2) plane with contours of f
x1g = np.linspace(-0.5, 1.2, 300)
x2g = np.linspace(-0.2, 1.2, 300)
X1, X2 = np.meshgrid(x1g, x2g)
F = f(X1, X2)

plt.figure(figsize=(7, 6))
cs = plt.contour(X1, X2, F, levels=30, cmap="viridis", alpha=0.6)
plt.plot(np.concatenate([[1.0], df_ada["x1"]]), np.concatenate([[1.0], df_ada["x2"]]),
         marker="o", color="tab:blue", label="AdaGrad", markersize=4)
plt.plot(np.concatenate([[1.0], df_rms["x1"]]), np.concatenate([[1.0], df_rms["x2"]]),
         marker="s", color="tab:red", label="RMSProp", markersize=4)
plt.scatter([0], [0], color="black", marker="*", s=150, zorder=5, label="Minimum (0,0)")
plt.xlabel("x1")
plt.ylabel("x2")
plt.title("Q3: Optimization Trajectories over Contours of f")
plt.legend()
plt.tight_layout()
plt.savefig("/home/claude/assignment2/plots/q3_trajectories.png", dpi=150)
plt.close()

print("\nFinal AdaGrad point:", (df_ada.iloc[-1]["x1"], df_ada.iloc[-1]["x2"]),
      "f =", df_ada.iloc[-1]["f(x1,x2)"])
print("Final RMSProp point:", (df_rms.iloc[-1]["x1"], df_rms.iloc[-1]["x2"]),
      "f =", df_rms.iloc[-1]["f(x1,x2)"])

```